Perf & FlameGraph

📌 时间：2023/6/19

作者：常亚辰

简介：本阶段成功运行了SPECjvm2008,并使用Compress测试了5个主流的JDK。尝试使用Perf、Perf-map-agent、async-profiler、FlameGraph等工具生成了火焰图。

1. 运行环境

主要通过一台Ubuntu 20.04的机器运行，具体运行环境如下表所示。

表1 运行环境

项目	内容
系统	Ubuntu 20.04
Kernel	5.15.0-74-generic
CPU	Intel I5-12400F
内存	32G

2. SPECjvm2008

2.1 全量测试

分别在Ubuntu和Windows11系统下进行了全量测试。因为JDK 1.8在运行startup.compiler.sunflow用例时阻塞，添加"-nowarn"后重新编译会导致checksum校验无法通过，所以结果是non compliant。如果使用JDK 1.7不会有这种问题。

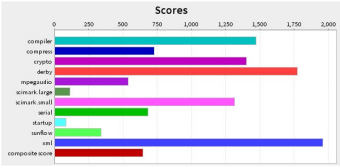
SPECjvm2008 Base

Noncompliant composite result: 644.61 ops/m

n/a n/a
Oracle Corporation Java HotSpot(TM) 64-Bit Server VM
Tested by: n/a
Test date: Wed May 31 17:53:58 CST 2023

Run is valid, but not compliant

Benchmark	ops/m
compiler	1471.58
compress	726.75
crypto	1400.9
derby	1772.44
mpegaudio	537.37
scimark.large	111.47
scimark.small	1315.21
serial	683.06
startup	84.09
sunflow	339.39
xml	1958.03
Noncompliant composite result: 644.61 ops/m	



run-info	
Submitter	n/a
Submitter URL	n/a
SPEC license	n/a
Tester	cyc
Location	n/a
Test date:	Wed May 31 17:53:58 CST 2023

sw-info	
OS name	Linux
OS available	n/a
OS address bits	n/a
OS tuning	
Filesystem	n/a
Other s/w name	
Other s/w tuning	
Other s/w available	

jvm-info	
Vendor	Oracle Corporation
Vendor URL	http://java.oracle.com/
JVM name	Java HotSpot(TM) 64-Bit Server VM
JVM version	25.371-b11 mixed mode
JVM available	n/a
Java Specification	1.8
JVM address bits	64
JVM initial heap memory	n/a
JVM maximum heap memory	n/a
JVM command line	n/a
JVM command line startup	
JVM launcher startup	default

hw-info	
HW vendor	n/a
HW vendor's URL	n/a
HW model	n/a
HW available	n/a
CPU vendor	n/a
CPU vendor's URL	n/a
CPU name	n/a
CPU frequency	n/a
# of logical cpus	n/a
# of chips	n/a
# of cores	n/a
Cores per chip	n/a
Threads per core	n/a

图1 Ubuntu20.04 Oracle JDK1.8u371 644.61ops/m

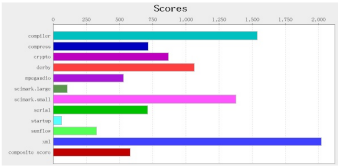
SPECjvm2008 Base

Composite result: 582.39 SPECjvm2008 Base ops/m

n/a n/a
Oracle Corporation Java HotSpot(TM) 64-Bit Server VM
Tested by: n/a
Test date: Mon Jun 05 22:15:14 CST 2023

Run is compliant

Benchmark	ops/m
compiler	1536.87
compress	717.19
crypto	870.64
derby	1062.91
mpegaudio	530.28
scimark.large	109.85
scimark.small	1376.06
serial	713.14
startup	67.57
sunflow	328.81
xml	2017.92
Composite result: 582.39 SPECjvm2008 Base ops/m	



run-info	
Submitter	n/a
Submitter URL	n/a
SPEC license	n/a
Tester	cyc
Location	n/a
Test date:	Mon Jun 05 22:15:14 CST 2023

sw-info	
OS name	Linux
OS available	n/a
OS address bits	n/a
OS tuning	
Filesystem	n/a
Other s/w name	
Other s/w tuning	
Other s/w available	

jvm-info	
Vendor	Oracle Corporation
Vendor URL	http://java.oracle.com/
JVM name	Java HotSpot(TM) 64-Bit Server VM
JVM version	24.80-b11 mixed mode
JVM available	n/a
Java Specification	1.7
JVM address bits	64
JVM initial heap memory	n/a
JVM maximum heap memory	n/a
JVM command line	n/a

hw-info	
HW vendor	n/a
HW vendor's URL	n/a
HW model	n/a
HW available	n/a
CPU vendor	n/a
CPU vendor's URL	n/a
CPU name	n/a
CPU frequency	n/a
# of logical cpus	n/a
# of chips	n/a

图2 Ubuntu20.04 Oracle JDK1.7u80 582.39ops/m

在Ubuntu20.04系统下，Oracle JDK1.8u371的分数为644.61ops/m，Oracle JDK1.7u80的分数为582.39 ops/m。其中JDK1.8 和JDK1.7在crypto和derby上差距比较明显，其中crypto差距最明显的workload是crypto.rsa，二者的分数分别为5390.14和1355.95，后面我尝试分析了性能差距的来源，具体分析见第三章的实践。

2.2 Compress用例下不同JDK对比

这里测试了几个主流的JDK在Compress用例下进行三轮测试的表现，数据如下：

表2 JDK在compress用例下的对比

JDK	Round1	Round2	Round3	Mean	

Oracle JDK 8-371	697.6	703.8	700.57	700.66
Open JDK 8-372	707.22	707.2	707.72	707.38
TecentKona JDK 8-322	706.73	706.63	708.19	707.18
BiSheng JDK 8-372	716.66	713.3	705.17	**711.71**
Alibaba Dragonwell JDK8-372	713.46	708.49	708.95	710.3

可以看到，主流的JDK在Compress用例下表现比较接近，其中Oracle JDK 8-371平均得分最低，为700.66。BiSheng JDK8-372最高，平均得分为711.71。由于仅进行了三轮测试，并且得分非常接近，我认为根据这些数据得到的排名不是很可靠，后续可以考虑更完善的对比实验。

3. Perf & Flame Graph

3.1 生成Flame Graph

分别通过Perf & Perf-map-agent & Flame Graph和async- profiler生成了火焰图，并对比了这两种方式生成火焰图对性能区别,由于时间原因只测试了一轮，Oracle JDK 8取表2里它的平均值。

表3 不同火焰图生成方法对性能的影响

JDK	ops/m	下降
Oracle JDK 8-371	700.6567	
+Perf-map-agent	651.64	下降约7%
+Perf-map-agen& +PreserveFramePointer	637.19	下降约9%
+async-profiler	698.92	下降约0.3%

可以看到，如果使用Perf，对性能的影响是很高的，通过sar -u命令，在使用Perf采样SPECjvm时，%user为91.7%，%system为7.81%，作为对比，与未使用perf时运行compress用例的%user为99.09%，%system为0.17%。使用Perf采样SPECjvm时CPU占用比例如下：

```
Processes
PID     Command                    %CPU▼  %MEM

 3561   java                       88.2   1.0
 3571   perf record                7.9    0.0
 3723   node                       0.8    0.3
 1635   systemd                    0.5    0.0
 1      init                       0.1    0.0
 3498   gnome-terminal-server      0.1    0.2
```

图3 SPECjvm时CPU占用比例

如果使用Perf-map-agen时不设置XX:+PreserveFramePointer保留帧指针的话，火焰图只会显示栈顶的函数，如图5所示，但开启后会额外带来性能损失，同时开启Perf-map-agen&PreserveFramePointer的得分下降了9%。我这里缺失了单独开启PreserveFramePointer的性能实验，不过根据Netflix的报告(https://netflixtechblog.com/java-in-flames-e763b3d32166)，性能损耗在 0 到3%左右。

async-profiler对性能的影响很小，应该和它使用的AsyncGetCallTrace有关，需要后续探索。生成的火焰图如下：

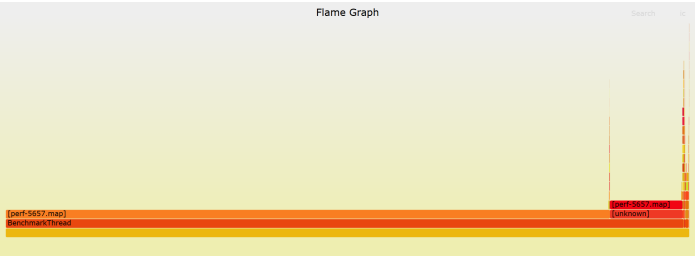


图4 只使用perf

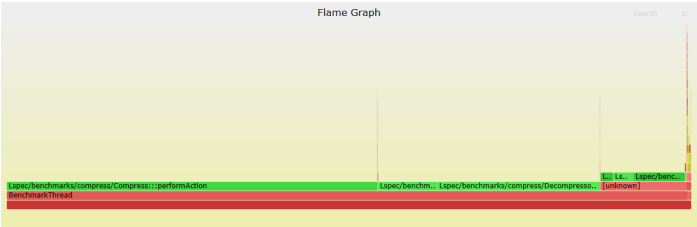


图5 使用Perf-map-agent

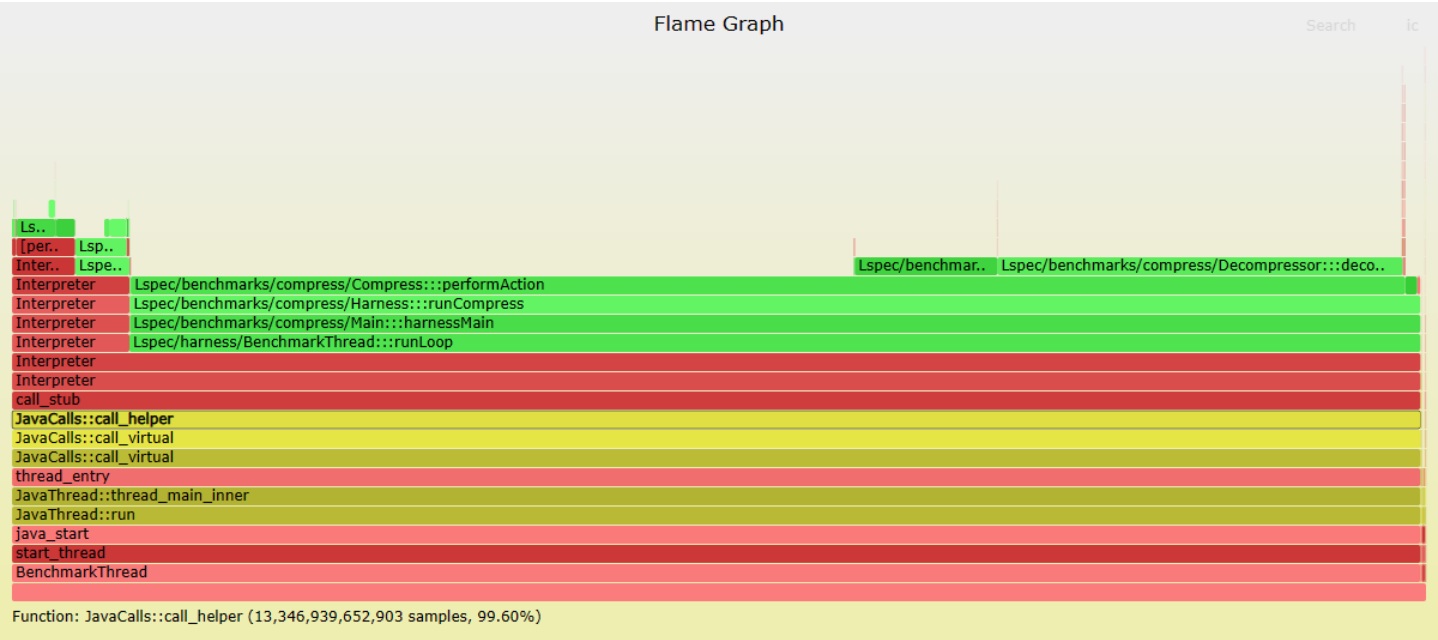


图6 使用perf map agent 并开启PreserveFramePointer

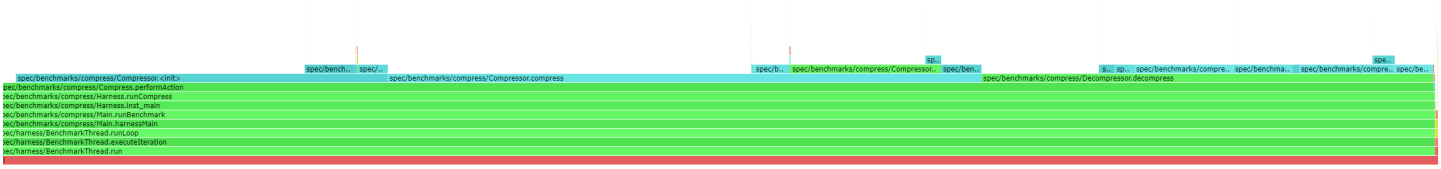


图7 使用async-profiler

3.2 性能分析尝试

在第2章提到Orcale JDK1.8 和JDK1.7在crypto和derby上差距比较明显，其中crypto差距最明显的workload是crypto.rsa，二者的分数分别为5390.14和1355.95。这里尝试探索二者性能差距的来源。

为了分析主要的性能区别，我通过async-profiler分别生成了JDK1.8和JDK1.7运行crypto.rsa的火焰图：

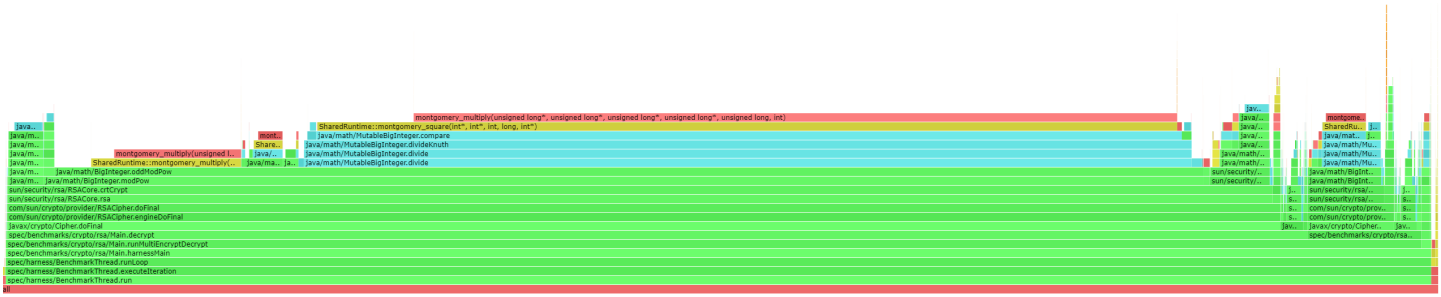


图8 Orcale JDK1.8运行crypto.rsa的火焰图

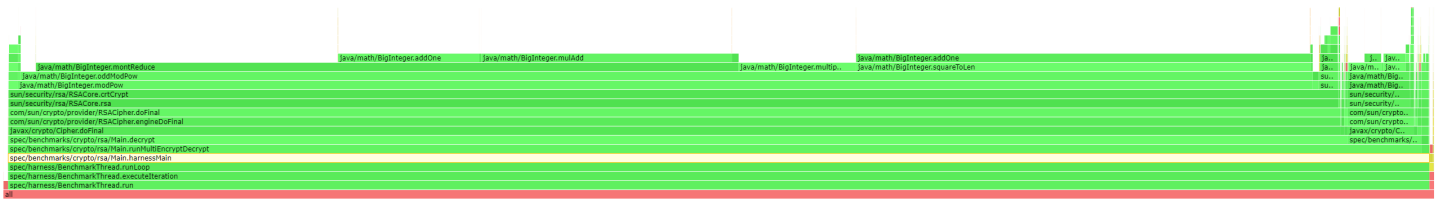


图9 Orcale JDK1.7运行crypto.rsa的火焰图

可以看到， BigInteger.modPow的运行时间占比从jdk1.7中的90%下降到JDK 1.8中的80%，并且二者的stack不太一致，因此我猜想性能差距的原因是不同JDK的BigInteger.modPow的性能不一样。为了验证我的猜想，我利用JMH仿照RSA的m、e、c使用modPow进行了实验，benchmark mode使用Throughput，其他均使用默认参数，结果如下：

表4 BigInteger.modPow性能对比

版本	Score(ops/s)	Errors
1.8	2904.053	4.780
1.7	741.030	19.919

根据火焰图上出现的montgomery_multiply,我们可以在https://bugs.openjdk.org/browse/JDK-8153189和https://bugs.openjdk.org/browse/JDK-8081778找到相关的改进。

JDK / JDK-8130150 Implement BigInteger.montgomeryMultiply intrinsic / JDK-8153189

Release Note: Implemented performance improvements for BigInteger.montgomeryMultiply

Details

Type:	Sub-task	Status:	CLOSED
Priority:	P4	Resolution:	Delivered
Affects Version/s:	7u97, 7u111, 8u77, 8u102, 9	Fix Version/s:	9
Component/s:	hotspot		
Labels:	NewFeature release-note		
Subcomponent:	compiler		
Verification:	Verified		

Backports

Issue	Fix Version	Assignee	Priority	Status	Resolution	Resolved In Build
JDK-8158936	8u102	Clifford Wayne	P4	Closed	Delivered	
JDK-8176307	8u77	Owen Stuart	P4	Closed	Delivered	
JDK-8157880	7u111	Clifford Wayne	P4	Closed	Delivered	
JDK-8176305	7u97	Owen Stuart	P4	Closed	Delivered	b33

Description

We have implemented improvements that will improve performance of several security algorithms, especially when using ciphers with key lengths of 2048-bit or greater. To turn on these improvements, use the options -XX:+UseMontgomeryMultiplyIntrinsic and -XX:+UseMontgomerySquareIntrinsic. This improvement is only for Linux and Solaris on x86_64 architecture.

Issue Links

图10 JDK release note 8130150

JDK / JDK-8081778

Use Intel x64 CPU instructions for RSA acceleration

Details

Type:	Enhancement	Status:	RESOLVED
Priority:	P3	Resolution:	Fixed
Affects Version/s:	9	Fix Version/s:	9
Component/s:	hotspot		
Labels:	release-note=no ursqe-has-regtest		
Subcomponent:	compiler		
Resolved In Build:	b70		
CPU:	x86		
OS:	generic		

Backports

Issue	Fix Version	Assignee	Priority	Status	Resolution	Resolved In Build
JDK-8135484	emb-9	Vladimir Kozlov	P3	Resolved	Fixed	team
JDK-8161779	8u112	Ivan Gerasimov	P3	Resolved	Fixed	b03
JDK-8156260	8u111	Ivan Gerasimov	P3	Resolved	Fixed	b01
JDK-8150024	8u102	Ivan Gerasimov	P3	Resolved	Fixed	b01
JDK-8153442	8u101	Ivan Gerasimov	P3	Resolved	Fixed	b02
JDK-8153423	8u92	Ivan Gerasimov	P3	Resolved	Fixed	b31
JDK-8152448	8u77	Ivan Gerasimov	P3	Resolved	Fixed	b31
JDK-8162057	emb-8u111	Ivan Gerasimov	P3	Resolved	Fixed	b01
JDK-8155359	emb-8u101	Ivan Gerasimov	P3	Resolved	Fixed	b02

Description

The optimization is achieved through intrinsics for BigInteger squareToLen and mulAdd methods.

We see ~50% improvement for SPECjvm2008 crypto.rsa with this patch.

图11 JDK release note 8081778

看起来是添加了一些关于montgomery multiply的intrinsic导致JDK1.8的modPow性能远超JDK1.7。如果我们关闭JDK1.8的这些intrinsic，那么JDK1.7和JDK1.8的性能会比较接近。

表5 关闭相关intrinsic前后性能对比

项目	版本	得分
JMH BigInteger.modPow	Oracle JDK1.8u371	2904.053
	1.8u371关闭intrinsic	853.419
	Oracle JDK 1.7u80	741.030
SPECjvm crypto.rsa	Oracle JDK 1.8u371	5390.14
	1.8u371关闭intrinsic	1493.99
	Oracle JDK 1.7u80	1355.95

4. 存在的问题

1. 使用perf-map-agent生成火焰图时，相同的脚本有时图像上显示unknown，有时正常显示为图6的状态。我的脚本如下：

```
set -m
export setting PERF_RECORD_SECONDS=360
java -XX:+PreserveFramePointer -jar SPECjvm2008.jar  -ikv compress &
spec_pid=$!
echo $spec_pid
perf-java-flames $spec_pid  -F 99 -a -g -p $spec_pid &
wait
```

生成的图像由两种情况，一种是下图，一种是图6。

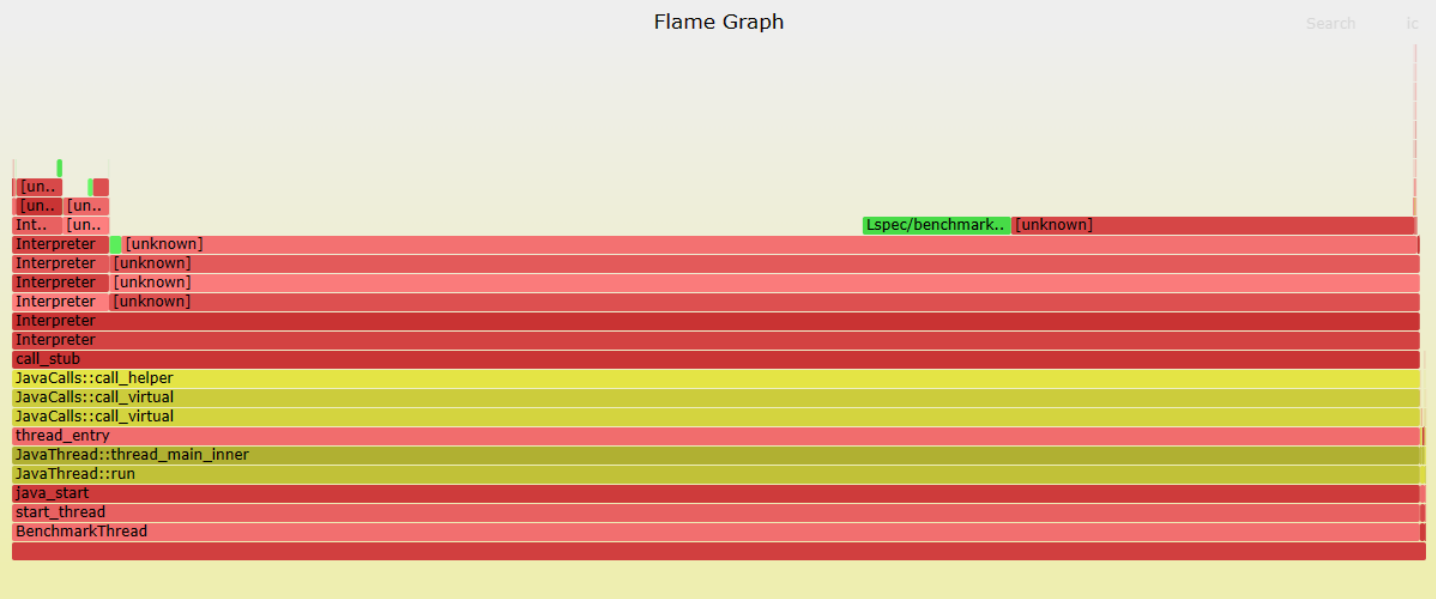


图12 perf-map-agent异常情况

2. 另外尝试使用GCViewer观察GC日志：

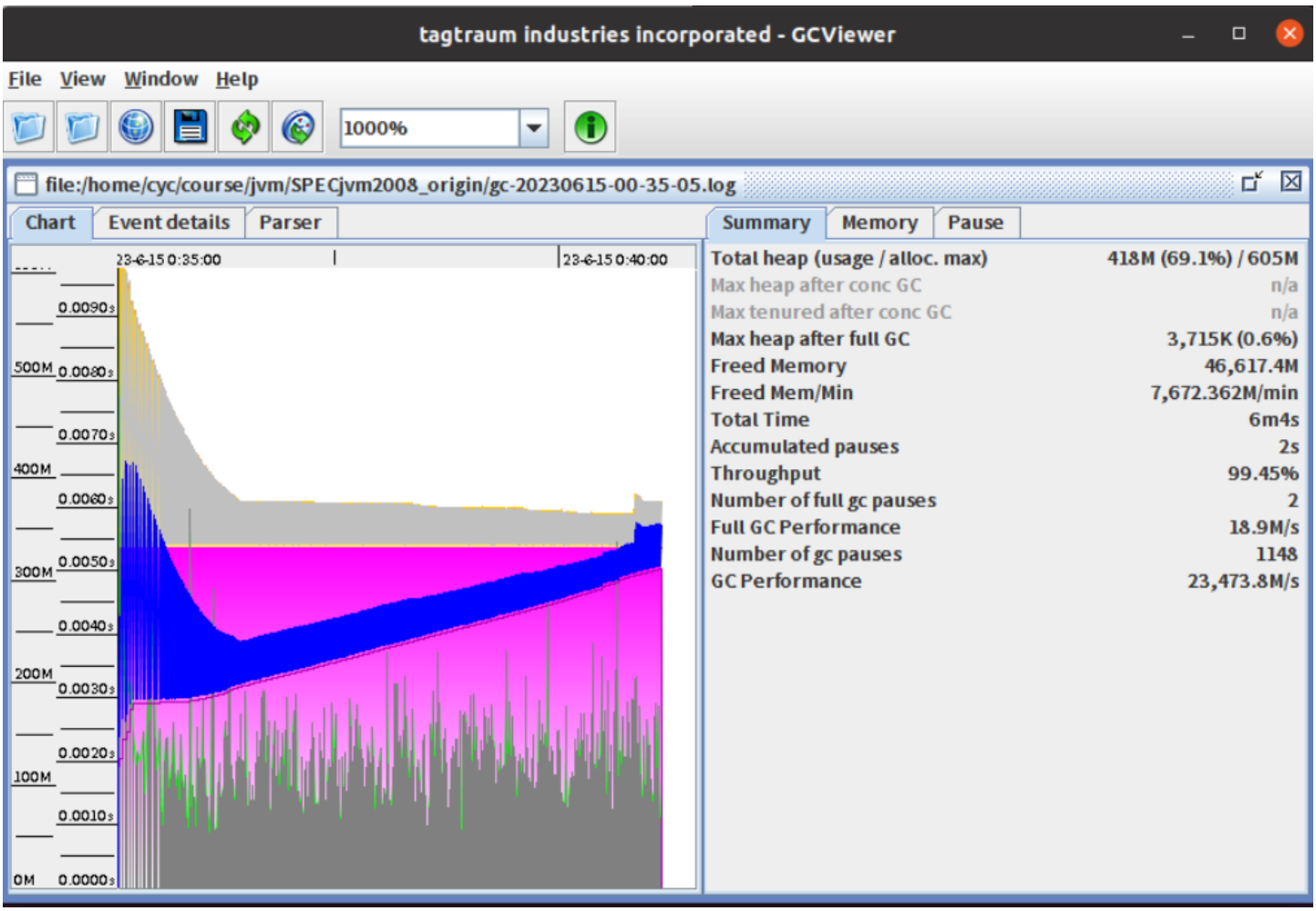


图3 GCViewer分析

但目前不知道应该注意哪些指标，另外arthas看起来是一个更全面的工具，后面可以尝试使用。